

Backend Engineering Specification

Full-Stack REST API & Monorepo Architecture Guide

Version 1.0 · May 2026

Confidential — Internal Engineering Document

Prepared for: Irfan (AI IDE Build)

Table of Contents

1.	Project Overview & Goals	3
2.	Monorepo Structure	4
3.	Tech Stack & Dependencies	5
4.	Environment Setup	6
5.	Database Schema (PostgreSQL)	7
6.	Authentication & Security	10
7.	REST API Reference — Gallery	11
8.	REST API Reference — Announcements	13
9.	REST API Reference — Tour Media	15
10.	REST API Reference — Auth	16
11.	File Upload Strategy	17
12.	Frontend ↔ Backend Integration	18
13.	Deployment Guide	19
14.	Error Codes & Responses	20
15.	Security Checklist	21

1. Project Overview & Goals

What We Are Building

Aarambh Pre-Primary School currently runs a fully static frontend (HTML/CSS/JS + localStorage). The goal is to replace localStorage with a real PostgreSQL-backed REST API so that gallery images, announcements, and tour media are stored server-side, persist across devices, and can be managed securely by authenticated admins.

Core Objectives

- Replace localStorage with a persistent PostgreSQL database
- Build a Node.js/Express REST API with full CRUD for Gallery, Announcements, and Tour Media
- Implement JWT-based admin authentication (replacing hardcoded credentials)
- Handle image/media file uploads via Cloudinary (free tier)
- Keep the existing frontend working with minimal JS changes (swap localStorage calls to fetch() calls)
- Deploy API on Railway or Render (free tier), frontend on Vercel or GitHub Pages

Scope — What is NOT in scope (v1)

- Admissions form processing (backend stub only)
- WhatsApp/Email notification system
- Multi-admin role management
- Real-time features (WebSockets)

2. Monorepo Structure

Use a single Git repository with two top-level workspaces. This keeps frontend and backend versioned together while allowing independent deployments.

```
aarambh/
├── package.json           # root – workspaces config
├── .gitignore
├── .env.example           # shared env template
├──
├── frontend/             # existing site (zero changes to HTML/CSS)
│   ├── index.html
│   ├── admin.html
│   ├── style.css
│   ├── admin.css
│   ├── script.js         # ← only this file is updated
│   ├── admin.js          # ← only this file is updated
│   └── img/
│   └──
├── backend/
│   ├── package.json
│   ├── .env              # never committed
│   ├── server.js         # entry point
│   ├── app.js            # express app setup
│   ├──
│   ├── config/
│   │   ├── db.js         # pg Pool / Sequelize setup
│   │   ├── cloudinary.js # cloudinary SDK config
│   │   └──
│   ├── middleware/
│   │   ├── auth.js       # verifyToken middleware
│   │   ├── upload.js     # multer + cloudinary storage
│   │   └── errorHandler.js # global error handler
│   │   └──
│   ├── models/
│   │   ├── User.js
│   │   ├── GalleryImage.js
│   │   ├── Announcement.js
│   │   └── TourMedia.js
│   │   └──
│   ├── routes/
│   │   ├── auth.js       # POST /api/auth/login
│   │   ├── gallery.js    # CRUD /api/gallery
│   │   ├── announcements.js # CRUD /api/announcements
│   │   └── media.js      # CRUD /api/media
```

```
■
■■■■ controllers/
■   ■■■■ authController.js
■   ■■■■ galleryController.js
■   ■■■■ announcementController.js
■   ■■■■ mediaController.js
■
■■■■ migrations/
    ■■■■ 001_init.sql      # initial schema
```

Root package.json (workspaces)

```
{
  "name": "aarambh",
  "private": true,
  "workspaces": ["frontend", "backend"],
  "scripts": {
    "dev": "cd backend && npm run dev",
    "start": "cd backend && npm start"
  }
}
```

3. Tech Stack & Dependencies

Layer	Technology	Version	Purpose
Runtime	Node.js	20 LTS	JavaScript server runtime
Framework	Express.js	^4.18	HTTP routing and middleware
Database	PostgreSQL	15+	Relational data storage
DB Client	pg (node-postgres)	^8.11	Raw SQL queries, connection pooling
Auth	jsonwebtoken + bcryptjs	^9 / ^2	JWT tokens, password hashing
File Upload	multer + cloudinary	latest	Multipart forms + cloud storage
Validation	express-validator	^7	Request body validation
CORS	cors	^2.8	Cross-origin requests from frontend
Env	dotenv	^16	Environment variable loading
Dev Tool	nodemon	^3	Auto-restart on file changes
Hosting (API)	Railway or Render	free	Cloud deployment for backend
Hosting (FE)	Vercel / GitHub Pages	free	Static site deployment
Storage	Cloudinary	free	Image/video CDN hosting

backend/package.json — dependencies block

```
{
  "dependencies": {
    "cloudinary": "^1.41.3",
    "cors": "^2.8.5",
    "dotenv": "^16.3.1",
    "express": "^4.18.2",
    "express-validator": "^7.0.1",
    "jsonwebtoken": "^9.0.2",
    "multer": "^1.4.5-lts.1",
    "multer-storage-cloudinary": "^4.0.0",
    "bcryptjs": "^2.4.3",
    "pg": "^8.11.3"
  },
  "devDependencies": {
    "nodemon": "^3.0.1"
  },
  "scripts": {
```

```
    "start": "node server.js",  
    "dev": "nodemon server.js"  
  }  
}
```

4. Environment Setup

backend/.env (never commit this file)

```
# Server
PORT=5000
NODE_ENV=development

# Database – get from Neon.tech (free Postgres) or Railway
DATABASE_URL=postgresql://user:password@host:5432/aarambh_db
# OR individual fields:
DB_HOST=localhost
DB_PORT=5432
DB_NAME=aarambh_db
DB_USER=postgres
DB_PASSWORD=yourpassword

# JWT
JWT_SECRET=replace_with_64_char_random_string
JWT_EXPIRES_IN=7d

# Admin credentials (hashed at seed time)
ADMIN_USERNAME=admin
ADMIN_PASSWORD=aarambh@2026 # plain for seed script only

# Cloudinary (get from cloudinary.com – free account)
CLOUDINARY_CLOUD_NAME=your_cloud_name
CLOUDINARY_API_KEY=your_api_key
CLOUDINARY_API_SECRET=your_api_secret

# Frontend URL (for CORS whitelist)
FRONTEND_URL=https://aarambh.vercel.app
```

config/db.js — PostgreSQL Pool

```
const { Pool } = require('pg');
const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
  ssl: process.env.NODE_ENV === 'production'
    ? { rejectUnauthorized: false }
    : false,
});
module.exports = pool;
```

app.js — Express Setup

```
const express = require('express');
const cors = require('cors');
```

```
const errorHandler = require('./middleware/errorHandler');
const app = express();
app.use(cors({
  origin: process.env.FRONTEND_URL || '*',
  credentials: true,
}));
app.use(express.json({ limit: '10mb' }));
app.use(express.urlencoded({ extended: true, limit: '10mb' }));
// Routes
app.use('/api/auth',      require('./routes/auth'));
app.use('/api/gallery',   require('./routes/gallery'));
app.use('/api/announcements', require('./routes/announcements'));
app.use('/api/media',     require('./routes/media'));
// Health check
app.get('/health', (_, res) => res.json({ status: 'ok' }));
app.use(errorHandler);
module.exports = app;
```

migrations/001_init.sql — Run once to create all tables

[illegible]

[illegible]

Seed Script — scripts/seed.js

```
require('dotenv').config();
const pool      = require('../config/db');
const bcrypt    = require('bcryptjs');
async function seed() {
```

```
const hash = await bcrypt.hash(process.env.ADMIN_PASSWORD, 12);
await pool.query(
  `INSERT INTO admins (username, password)
  VALUES ($1, $2)
  ON CONFLICT (username) DO NOTHING`,
  [process.env.ADMIN_USERNAME, hash]
);
console.log('Seed complete.');
```

```
process.exit(0);
}
seed().catch(err => { console.error(err); process.exit(1); });
```

6. Authentication & Security

Flow

- Admin POSTs username + password to POST /api/auth/login
- Server looks up admin in DB, compares bcrypt hash
- On success: returns signed JWT (7-day expiry)
- Admin stores token in sessionStorage (existing admin.js pattern)
- All write endpoints (POST/PUT/DELETE) require Authorization: Bearer <token> header
- Public read endpoints (GET gallery, GET announcements, GET media) — NO auth required

middleware/auth.js

```
const jwt = require('jsonwebtoken');
module.exports = function verifyToken(req, res, next) {
  const header = req.headers['authorization'];
  if (!header || !header.startsWith('Bearer '))
    return res.status(401).json({ error: 'No token provided' });
  const token = header.split(' ')[1];
  try {
    req.admin = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch {
    return res.status(403).json({ error: 'Invalid or expired token' });
  }
};
```

controllers/authController.js

```
const pool = require('../config/db');
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
exports.login = async (req, res, next) => {
  try {
    const { username, password } = req.body;
    const { rows } = await pool.query(
      'SELECT * FROM admins WHERE username = $1', [username]
    );
    if (!rows.length) return res.status(401).json({ error: 'Invalid credentials' });
    const valid = await bcrypt.compare(password, rows[0].password);
    if (!valid) return res.status(401).json({ error: 'Invalid credentials' });
    const token = jwt.sign(
```

```
    { id: rows[0].id, username: rows[0].username },  
    process.env.JWT_SECRET,  
    { expiresIn: process.env.JWT_EXPIRES_IN }  
  );  
  res.json({ token, username: rows[0].username });  
} catch (err) { next(err); }  
};
```

7. REST API Reference — Gallery

Method	Endpoint	Description	Auth
GET	/api/gallery	Get all images (sorted by sort_order)	Public
GET	/api/gallery/:id	Get single image by UUID	Public
POST	/api/gallery	Upload new image (multipart/form-data)	Admin JWT
PUT	/api/gallery/:id	Update title / category	Admin JWT
PATCH	/api/gallery/reorder	Bulk update sort_order array	Admin JWT
DELETE	/api/gallery/:id	Delete image (removes from Cloudinary too)	Admin JWT

POST /api/gallery — Request (multipart/form-data)

Field	Type	Required	Description
image	File	Yes	Image file (jpg/png/webp, max 5MB)
title	string	Yes	Alt text / display name of the image
category	string	Yes	One of: classroom activity event

Success Response — 201 Created

```
{
  "id": "a1b2c3d4-...",
  "title": "Bright Classroom",
  "category": "classroom",
  "url": "https://res.cloudinary.com/.../gallery/abc123.jpg",
  "cloudinary_id": "aarambh/gallery/abc123",
  "sort_order": 7,
  "created_at": "2026-05-23T10:00:00Z"
}
```

PATCH /api/gallery/reorder — Request Body

```
{
  "order": [
    { "id": "uuid-1", "sort_order": 0 },
    { "id": "uuid-2", "sort_order": 1 },
    { "id": "uuid-3", "sort_order": 2 }
  ]
}
```

routes/gallery.js

```
const router    = require('express').Router();
const ctrl      = require('../controllers/galleryController');
const auth      = require('../middleware/auth');
const { upload } = require('../middleware/upload');

router.get('/',          ctrl.getAll);
router.get('/:id',       ctrl.getOne);
router.post('/',         auth, upload.single('image'), ctrl.create);
router.put('/:id',      auth, ctrl.update);
router.patch('/reorder', auth, ctrl.reorder);
router.delete('/:id',auth, ctrl.remove);

module.exports = router;
```

controllers/galleryController.js — key methods

```
const pool = require('../config/db');
const cloudinary = require('../config/cloudinary');

exports.getAll = async (req, res, next) => {
  try {
    const { rows } = await pool.query(
      'SELECT * FROM gallery_images ORDER BY sort_order ASC'
    );
    res.json(rows);
  } catch (err) { next(err); }
};

exports.create = async (req, res, next) => {
  try {
    const { title, category } = req.body;
    // req.file.path and req.file.filename come from multer-cloudinary
    const { rows } = await pool.query(
      `INSERT INTO gallery_images
        (title, category, cloudinary_id, url, sort_order)
        VALUES ($1,$2,$3,$4,
          (SELECT COALESCE(MAX(sort_order)+1,0) FROM gallery_images))
        RETURNING *`,
      [title, category, req.file.filename, req.file.path]
    );
    res.status(201).json(rows[0]);
  } catch (err) { next(err); }
};

exports.remove = async (req, res, next) => {
  try {
    const { rows } = await pool.query(
      'DELETE FROM gallery_images WHERE id=$1 RETURNING cloudinary_id',

```

```
    [req.params.id]
  );
  if (!rows.length) return res.status(404).json({ error: 'Not found' });
  await cloudinary.uploader.destroy(rows[0].cloudinary_id);
  res.json({ message: 'Deleted' });
} catch (err) { next(err); }
};
```

8. REST API Reference — Announcements

Method	Endpoint	Description	Auth
GET	/api/announcements	Get all announcements (pinned first)	Public
GET	/api/announcements/:id	Get single announcement	Public
POST	/api/announcements	Create announcement (optional image upload)	Admin JWT
PUT	/api/announcements/:id	Update announcement	Admin JWT
DELETE	/api/announcements/:id	Delete announcement	Admin JWT

POST /PUT /api/announcements — Request Schema

Field	Type	Required	Description
title	string	Yes	Announcement title (max 300 chars)
message	string	Yes	Body text
ann_date	date	Yes	ISO date string: YYYY-MM-DD
status	string	Yes	One of: New Event Alert Holiday
pinned	boolean	No	Default false. Only one should be pinned.
btn_text	string	No	CTA button label
btn_link	string	No	CTA button href
image	File	No	Optional announcement image (multipart)

GET /api/announcements — Response

```
[
  {
    "id": "uuid",
    "title": "Admissions open 2026-27",
    "message": "Early bird discounts available...",
    "ann_date": "2026-06-01",
    "status": "New",
    "pinned": true,
    "btn_text": "Apply Now",
    "btn_link": "#admissions",
    "image_url": "https://res.cloudinary.com/.../ann_img.jpg",
    "cloudinary_id": "aarambh/announcements/xyz",
    "created_at": "2026-05-20T09:00:00Z",
```

```
    "updated_at": "2026-05-20T09:00:00Z"  
  },  
  ...  
]
```

9. REST API Reference — Tour Media

Method	Endpoint	Description	Auth
GET	/api/media	Get current active tour media	Public
POST	/api/media	Create/replace tour media record	Admin JWT
PUT	/api/media	Update title, description, src	Admin JWT
DELETE	/api/media	Soft-delete (sets is_deleted=true)	Admin JWT

POST /api/media — Request Schema

Field	Type	Required	Description
title	string	Yes	Section heading
description	string	No	Subtitle / description text
media_type	string	Yes	video audio
src_type	string	Yes	youtube mp4 audio-url cloudinary
src	string	Yes	YouTube ID, direct URL, or Cloudinary public_id
thumbnail_url	string	No	Cover image URL for audio or video poster
media_file	File	No	Optional upload (stored to Cloudinary)

GET /api/media — Response

```
{
  "id": "uuid",
  "title": "School Tour",
  "description": "Discover our beautiful campus...",
  "media_type": "video",
  "src_type": "youtube",
  "src": "dQw4w9WgXcQ",
  "thumbnail_url": null,
  "is_deleted": false,
  "created_at": "...",
  "updated_at": "..."
}
```

10. REST API Reference — Auth

Method	Endpoint	Description	Auth
POST	/api/auth/login	Authenticate admin, return JWT	Public
GET	/api/auth/verify	Verify JWT still valid	Admin JWT

POST /api/auth/login

Field	Type	Required	Description
username	string	Yes	Admin username
password	string	Yes	Admin password (plain — HTTPS only)

200 OK Response

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "username": "admin"
}
```

401 Unauthorized Response

```
{ "error": "Invalid credentials" }
```

11. File Upload Strategy

All images/media are uploaded directly to Cloudinary using `multer-storage-cloudinary`. The server never stores files on disk — files stream directly to Cloudinary and the returned URL is saved in PostgreSQL.

`config/cloudinary.js`

```
const cloudinary = require('cloudinary').v2;
cloudinary.config({
  cloud_name: process.env.CLOUDINARY_CLOUD_NAME,
  api_key:    process.env.CLOUDINARY_API_KEY,
  api_secret: process.env.CLOUDINARY_API_SECRET,
});
module.exports = cloudinary;
```

`middleware/upload.js`

```
const multer = require('multer');
const { CloudinaryStorage } = require('multer-storage-cloudinary');
const cloudinary = require('../config/cloudinary');
const storage = new CloudinaryStorage({
  cloudinary,
  params: async (req, file) => {
    let folder = 'aarambh/gallery';
    if (req.baseUrl.includes('announcements')) folder = 'aarambh/announcements';
    if (req.baseUrl.includes('media')) folder = 'aarambh/media';
    return {
      folder,
      allowed_formats: ['jpg', 'jpeg', 'png', 'webp', 'mp4', 'mp3'],
      resource_type: 'auto',
      transformation: [{ quality: 'auto', fetch_format: 'auto' }],
    };
  },
});
const upload = multer({
  storage,
  limits: { fileSize: 10 * 1024 * 1024 }, // 10MB max
});
module.exports = { upload };
```

Cloudinary Folder Structure

aarambh/

gallery/ ← gallery images

announcements/ ← announcement optional images

media/ ← tour video/audio files

12. Frontend ↔ Backend Integration

The frontend JS files need minimal changes. Replace `localStorage.getItem` / `setItem` calls with `fetch()` calls to the API. The API base URL should be stored in a single constant.

script.js — Replace `loadGallery()`

```
const API = 'https://aarambh-api.railway.app'; // change to your deployed URL

async function loadGallery() {
  if (!galleryGrid) return;
  const res = await fetch(`${API}/api/gallery`);
  const data = await res.json();
  galleryGrid.innerHTML = '';
  data.forEach(item => {
    const div = document.createElement('div');
    div.className = `gallery-item ${item.category} show`;
    div.innerHTML = `
      
      <div class="overlay"><i class="fa-solid fa-magnifying-glass-plus"></i>
    </div>`;
    div.addEventListener('click', () => {
      lightboxImg.src = item.url;
      lightbox.style.display = 'flex';
    });
    galleryGrid.appendChild(div);
  });
  bindFilters();
}
```

admin.js — Replace `save-btn` handler

```
document.getElementById('save-btn').addEventListener('click', async () => {
  const token = sessionStorage.getItem('aarambh_token');
  // Reorder
  const order = galleryData.map((img, i) => ({ id: img.id, sort_order: i }));
  ;
  await fetch(`${API}/api/gallery/reorder`, {
    method: 'PATCH',
    headers: { 'Content-Type': 'application/json',
      'Authorization': `Bearer ${token}` },
    body: JSON.stringify({ order }),
  });
  alert('Gallery saved!');
});
```

admin.js — Replace login handler

```
loginForm.addEventListener('submit', async (e) => {
  e.preventDefault();
  const res = await fetch(`${API}/api/auth/login`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      username: document.getElementById('username').value,
      password: document.getElementById('password').value,
    }),
  });
  if (res.ok) {
    const { token } = await res.json();
    sessionStorage.setItem('aarambh_token', token);
    // show dashboard...
  } else {
    errorMsg.textContent = 'Invalid credentials';
    errorMsg.style.display = 'block';
  }
});
```

13. Deployment Guide

Step 1 — Database (Neon.tech — Free PostgreSQL)

- Go to neon.tech → Create account → New Project → "aarambh"
- Copy the connection string (DATABASE_URL)
- Open the SQL editor and paste + run migrations/001_init.sql
- Run seed: DATABASE_URL=... node backend/scripts/seed.js

Step 2 — Cloudinary (Free Media Storage)

- Go to cloudinary.com → Create free account
- Dashboard → copy Cloud Name, API Key, API Secret → paste into .env

Step 3 — Backend (Railway)

- Push monorepo to GitHub
- railway.app → New Project → Deploy from GitHub → select repo
- Set Root Directory: backend
- Add all .env variables in Railway Variables tab
- Railway auto-detects Node and deploys. Copy the generated URL.

Step 4 — Frontend (Vercel)

- vercel.com → Import Git Repository
- Set Root Directory: frontend
- No build command needed (static site)
- Edit script.js and admin.js: set const API = "https://your-railway-url.app"
- Redeploy

Step 5 — CORS Update

- In Railway Variables, set FRONTEND_URL = <https://your-vercel-url.vercel.app>
- This allows the frontend to call the API without CORS errors

14. Error Codes & Standard Responses

Code	Meaning	When It Occurs
200	OK	Successful GET, PUT, PATCH
201	Created	Successful POST (new resource)
400	Bad Request	Missing required fields, validation failed
401	Unauthorized	No token, wrong credentials
403	Forbidden	Token invalid or expired
404	Not Found	Resource ID does not exist in DB
409	Conflict	Duplicate username, unique constraint violated
413	Payload Too Large	File exceeds 10MB limit
500	Internal Server Error	Unhandled exception — check Railway logs

middleware/errorHandler.js

```
module.exports = function errorHandler(err, req, res, next) {  
  console.error('[ERROR]', err.message);  
  const status = err.statusCode || err.status || 500;  
  res.status(status).json({  
    error: err.message || 'Internal server error',  
    ...(process.env.NODE_ENV === 'development' && { stack: err.stack })  
  });  
};
```

15. Security Checklist

JWT_SECRET	Use at least 64 random characters. Generate with: <code>node -e "console.log(require('crypto').randomBytes(64).toString('hex'))"</code>
bcrypt rounds	Use cost factor 12 for bcrypt hashing (already in seed script)
HTTPS only	Railway and Vercel provide HTTPS by default. Never send passwords over HTTP.
CORS	Set <code>FRONTEND_URL</code> to exact production URL — avoid wildcard (*) in production
.env	Never commit .env to Git. Add to .gitignore. Use platform environment variables.
Rate limiting	Add <code>express-rate-limit</code> to <code>/api/auth/login</code> (max 10 req/15min)
File validation	<code>multer</code> already checks <code>fileSize</code> . Add <code>mime-type</code> check in <code>upload.js</code>
SQL injection	Always use parameterized queries (<code>\$1</code> , <code>\$2</code>). Never string-concatenate SQL.
Helmet.js	<code>npm install helmet</code> and add <code>app.use(require("helmet")())</code> in <code>app.js</code>
Token expiry	<code>JWT_EXPIRES_IN = 7d</code> . Implement token refresh if sessions need to last longer.

Document prepared by the Aarambh Dev Team · May 2026 · v1.0 — Hand this to Irfan along with the AI IDE Prompts file.